Key-Value Stores Explained

Redis vs DynamoDB vs etcd

Watch the 4-min video

In one line: a key-value store is a giant distributed dictionary: put, get and delete by key, in constant time. Use it when you always know the key; pick Redis for in-memory speed, DynamoDB for durable managed scale, and etcd for small, strongly consistent config.

1 The simplest data model

Every time a shopper adds a sneaker to their cart, your app must save the cart and read it back in milliseconds, for millions of shoppers. No joins, no reports. Just: *give me the cart for user 42*. That is the perfect job for a key-value store.

A key-value store is a **giant, distributed dictionary**. Put a value under a key. Get it back by the key. Delete it. The value is **opaque** to the store, or only lightly structured. Lookups are **constant time**, and that simplicity is what makes these stores so fast and so easy to scale.

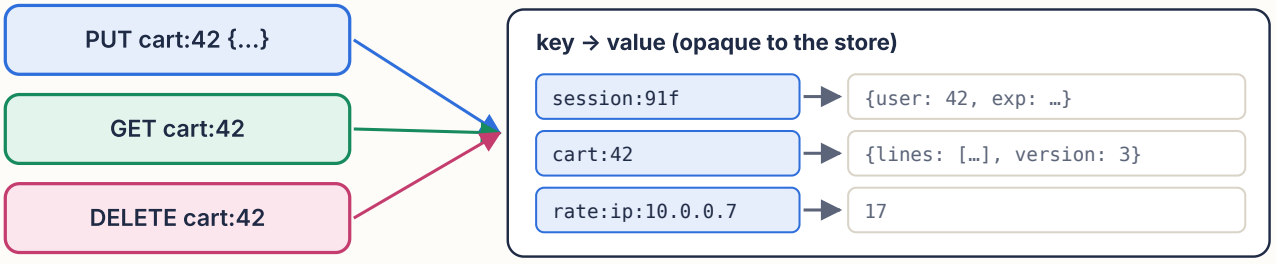


Figure 1. Three operations cover almost everything. The store does not look inside the value; it only finds it by key.

2 How it scales: hash partitioning

To go beyond one machine, keys are **hash partitioned**. Hash the key, and the result picks the partition, and the node that holds it. Plain `hash mod N` would move almost every key when N changes, so real systems use **consistent hashing** or a **fixed set of virtual partitions**: adding a node moves only a small fraction of the keys. Each partition is **replicated** for availability.

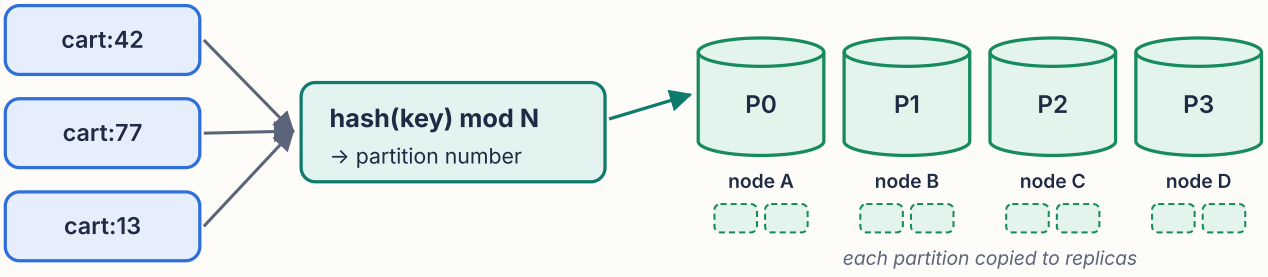


Figure 2. The hash spreads keys evenly, so load spreads evenly. Each partition lives on one node and is copied to replicas on others.

Two catches. A single **hot key** (one celebrity's counter, one flash-sale item) still lands on one partition, however many nodes you add. And **scanning a range of keys** is expensive, because hashing scatters

neighbouring keys across all partitions.

3 Time to live: expiry without a cleanup job

Many key-value stores let you set a **time to live (TTL)** on each key. The store deletes the key when it expires, so you need no cleanup job at all.

Data	Typical TTL	Why
Session	30 minutes	log the user out after inactivity
Shopping cart	30 days	abandoned carts clean themselves up
Rate limit counter	1 minute	the window resets on its own

Redis never returns an expired key: it deletes it when read and also samples keys in the background, so expired keys stop using memory soon. DynamoDB's TTL is a background process, so an expired item can linger for a while: filter on the expiry attribute when you read.

4 Example: a shopping cart in DynamoDB

The **partition key is the user ID**. The item holds the cart lines, a **version number**, and an **expiry attribute** for the TTL. Reading the cart is one `GetItem` call, in single-digit milliseconds. Adding an item is an `UpdateItem` **with a condition on the version**, so two devices don't overwrite each other. DynamoDB splits partitions automatically, and with on-demand mode you pay per request.

```
UpdateItem  Key = { user_id: "u42" }
  UpdateExpression    SET lines.#sku = :line,
                          version = version + :one,
                          expires_at = :now_plus_30d
  ConditionExpression version = :expected     -- 3
-- success → version is now 4
-- ConditionalCheckFailedException → someone else wrote first
```

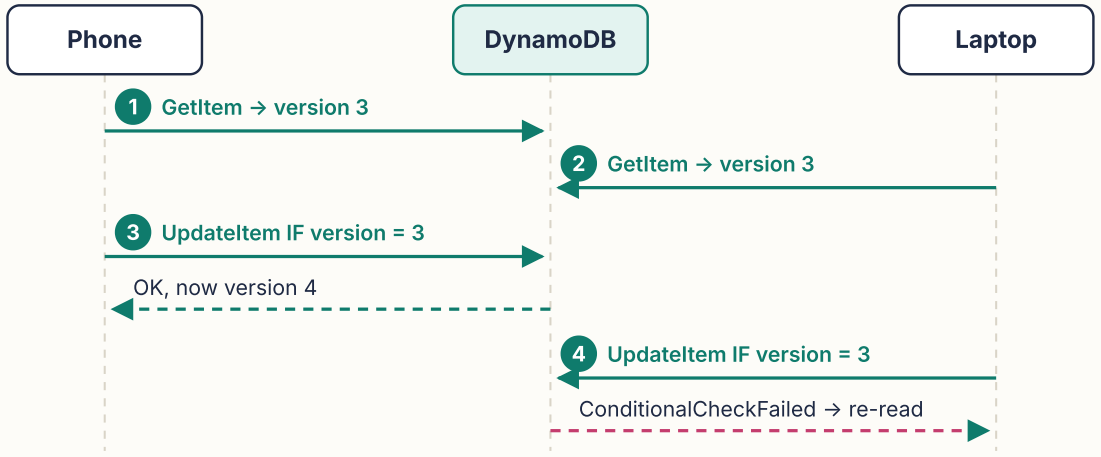


Figure 3. Both devices read version 3. The phone's write wins and bumps it to 4, so the laptop's write is refused instead of silently erasing the phone's sneaker.

WORKED EXAMPLE: SIZING THE CART STORE

- **Data:** 5 million active carts × about 2 KB each = **10 GB**. Tiny for DynamoDB (and well under its 400 KB per-item limit).
- **Traffic:** at peak, 20,000 cart reads/s and 4,000 writes/s. Random user IDs spread these across partitions, so no single partition is near its limit (about 3,000 reads and 1,000 writes per second each).
- **Cleanup:** a 30-day TTL deletes abandoned carts, so the table stops growing without a batch job.

5 Redis vs DynamoDB vs etcd

	Redis	DynamoDB	etcd
Storage	in memory, optional persistence	managed, durable, disk-backed	small, replicated log on disk
Speed	microseconds	single-digit milliseconds	milliseconds (consensus write)
Consistency	async replicas	eventual or strong reads	strong, built on Raft
Extras	lists, sets, sorted sets, counters	scales horizontally, no servers	watches, leases
Use it for	caches, sessions, leaderboards, rate limits	carts, profiles, durable data at scale	config, service discovery, leader election

etcd is **not for big data**: it is designed for a few gigabytes of critical metadata (Kubernetes keeps its cluster state there).

6 When it fits, and when it doesn't

Fits when you always know the key: sessions, carts, user preferences, feature flags, caches, rate limit counters and idempotency keys.

Poor fit when you need to query by other attributes, filter, sort, join or aggregate. You would end up maintaining your own secondary indexes, or you should pick a different database.

COMMON MISTAKES

- Storing data you later need to search by other fields (“all carts containing sneaker X”).
- Read-modify-write on a value with no version condition: two devices overwrite each other.
- Using Redis as the only copy of data you cannot lose, with persistence turned off.
- Putting large or high-volume data in etcd.
- Forgetting the hot key: one viral item can melt a partition.

7 Key terms

Key-value store: a distributed dictionary: put, get, delete

Consistent hashing: adding a node moves few keys

Conditional write: write only if a condition holds

Opaque value: the store does not look inside it

Hot key: one key with far more traffic than others

Raft: consensus protocol behind etcd

Hash partitioning: hash of the key picks the partition

TTL: automatic expiry of a key

Idempotency key: remembers a request so retries do not repeat it

8 Interview questions

Q1. What workloads fit a key-value store? BASIC
High-volume lookups by a known key with simple values and no joins: sessions, carts, caches, user preferences, feature flags, rate limit counters and idempotency keys, often with a TTL.

Q2. Why are key-value stores so easy to scale? BASIC
Every operation names one key. Hashing the key picks one partition, so requests spread evenly and nodes never need to coordinate for a lookup. Add nodes and move a small share of the keys.

Q3. Redis, DynamoDB or etcd for service discovery and leader election? BASIC
etcd. It is strongly consistent (Raft), offers watches and leases, and is built for small, critical metadata, not big data.

Q4. Two devices add items to the same cart at once. How do you stop lost updates? SCENARIO
Store a version in the item. UpdateItem with ConditionExpression version = :expected. The loser gets ConditionalCheckFailed, re-reads the cart and retries. Or use per-line updates (SET lines.#sku) so different items never conflict.

Q5. You need a global leaderboard of 10 million players, updated 50,000 times a second. SCENARIO
Redis sorted set: ZINCRBY to update a score and ZREVRANGE for the top 100, each in O(log N). Keep the durable score history in a database; Redis serves the ranking.

Q6. Product now wants "all carts that contain item X". What do you do? SCENARIO
A key-value store cannot answer that cheaply; it would scan every cart. Add a secondary index (a DynamoDB GSI on a per-line item, or a separate item_id → carts table), or stream changes to a database that can query it.

Q7. Adding nodes doesn't help one celebrity's like counter. Why, and what helps? TRICKY
One key always hashes to one partition. Split it: write to like:123#0 ... like:123#9 at random and sum the 10 on read, or count in Redis and flush to the store every second.

Q8. Why not use hash(key) mod N to pick the node? TRICKY
Change N from 10 to 11 and about 90% of keys move. Consistent hashing or a fixed set of virtual partitions (say 1,024 mapped to nodes) moves only about 1/N of the keys.

Next: SD-E-015 · Document Databases Explained backendroadmap.com